

FMCG eCommerce — Backend Feasibility Report

For Client Presentation | Solo Developer + AI | 1-Week Timeline

Part 1: Monolithic vs Microservices — Which Is Better?

Quick Answer: Monolithic is the right choice for you right now.

Factor	Monolithic	Microservices
Team size	Perfect for 1 developer	Needs 6–10 engineers
Build time	Days to weeks	4–6 months minimum
Complexity	Simple — one codebase	Extremely complex — 12 separate services
Infrastructure	One server, one DB	Kubernetes, Kafka, 5+ databases
Cost (hosting)	₹500–₹2,000/month	₹20,000–₹50,000/month (AWS EKS)
Debugging	Easy	Very hard (distributed tracing needed)
Deployment	One JAR file	12 Docker containers + orchestration
Integration with billing	Easy — same codebase/DB	Complex — inter-service REST + Kafka
AI assistance	Very effective	AI loses context across 12 repos

❑ The Truth About Microservices

The spec document defines microservices for a **team of 9 people over 12 months**. Netflix, Amazon, Uber use microservices — but they have **500+ engineers**. For a 1-week delivery by a solo developer, microservices would mean **you deliver 0 working features** and spend the whole week on infrastructure.

❑ Decision: Spring Boot Monolith

Same stack as your billing software. Can be split into microservices **later** if the business grows. This is called "**Modular Monolith**" — the industry best practice for startups.

Part 2: Complete Frontend vs Backend Feasibility Analysis

I analyzed every single page and service file in the frontend. Below is a **feature-by-feature breakdown** of what is achievable in 1 week.

🌀 IMPLEMENTABLE — Week 1 (Core APIs)

These directly power the frontend and are achievable in 1 week.

1. 🗄 Authentication

Frontend pages: Login, OTP screen, session management **What I'll build:**

- User registration (email/mobile + name)
- OTP-based login (email OTP via Spring Mail)
- JWT access token + refresh token
- Logout (token revocation)
- Role-based access (Customer / Admin)

Skipped: WhatsApp OTP, Google/Apple/Facebook OAuth (complex 3rd party)

2. 📦 Product Catalog

Frontend pages: Home page, Category page, Product detail [`id`], Search page **What I'll build:**

- List products (with filters: category, brand, price range, status)
- Get product by ID (with images, variants)
- Search products (by name, SKU, barcode)
- List categories (tree structure)
- Get product by barcode
- Compare products (up to 4)

Skipped: Elasticsearch full-text NLP search (complex), AI recommendations

3. 🛒 Cart

Frontend pages: Cart page, Cart drawer/sidebar **What I'll build:**

- Get user's cart
 - Add item to cart
 - Update item quantity
 - Remove item from cart
 - Apply coupon code
 - Remove coupon
 - Cart totals (subtotal, discount, delivery fee, tax, total)
-

4. 📦 Orders

Frontend pages: Checkout page, Order detail, Order history (account/orders) **What I'll build:**

- Create order (full checkout flow)
- List customer's orders
- Get order detail (items, timeline, status)
- Cancel order
- Reorder (prefill cart from past order)
- Order status history/timeline

Skipped: Live GPS tracking (needs real delivery partner app), Payment gateway (COD + stub for now)

5. 👤 Customer Profile & Addresses

Frontend pages: account/profile, account/addresses, account/settings **What I'll build:**

- Get own profile
 - Update profile (name, email, mobile)
 - Add/update/delete delivery address
 - Set default address
 - User preferences (dietary tags, allergens)
-

6. 🏢 Admin — Product Management

Frontend pages: admin/products, admin/products/categories, admin/products/pricing **What I'll build:**

- Create product
- Update product
- Delete product
- List products with search/filter/pagination
- Create/update/delete categories
- Update product pricing (price, MRP, cost, tax rate)
- Bulk product import (CSV/Excel — already in billing software!)

Skipped: admin/products/seo, admin/products/media (complex image CDN), admin/products/audit-logs

7. 🏢 Admin — Orders Management

Frontend pages: admin/orders, admin/orders/status-management **What I'll build:**

- List all orders (with filters: status, date range, customer)
- Get order detail
- Update order status (pending → confirmed → preparing → delivered)
- Add order notes
- Order summary counts (by status)

Skipped: admin/orders/assign-partner, admin/orders/substitutions, admin/orders/bulk-processing, admin/orders/timeline

8. 🏭 Admin — Inventory Management

Frontend pages: admin/inventory, admin/inventory/warehouses **What I'll build:**

- View stock levels per product/warehouse
- Adjust stock (add/deduct manually)
- Stock movement history
- Low stock alerts
- Warehouse list

Skipped: admin/inventory/forecast (ML), admin/inventory/fefo (complex batch), admin/inventory/stock-transfers, admin/inventory/safety-stock

9. 🏠 Admin — Customer Management

Frontend pages: admin/customers **What I'll build:**

- List all customers (with search/filter)
- View customer detail (orders, spend)
- Block/unblock customer
- Add customer notes

Skipped: admin/customers/analytics, admin/customers/segmentation, admin/customers/fraud-detection, admin/customers/support-tickets

10. 🏢 Admin — Dashboard

Frontend pages: admin/dashboard **What I'll build:**

- Revenue summary (total, growth, chart data)

- Orders summary (total, pending, by status)
 - Customers summary (total, active)
 - Top selling products
 - Low stock alerts
 - Recent/live orders feed
-

11. Promotions & Coupons (Basic)

Frontend pages: admin/promotions, admin/promotions/coupons **What I'll build:**

- List active promotions
- Create/update/delete promotion
- Coupon CRUD (code, type, discount, validity, min order, max uses)
- Validate coupon at checkout

Skipped: admin/promotions/flash-sales, admin/promotions/campaign-builder, admin/promotions/ab-testing, admin/promotions/push-notifications

PARTIAL — Limited Implementation

These will work but with reduced features.

12. Loyalty / Points System

Frontend pages: account/page (loyalty tab), loyalty-store.ts **What I'll build (partially):**

- Loyalty account (points balance, tier: Silver/Gold/Platinum)
- Points earned on order completion
- Points transaction history
- Burn points at checkout (skip for week 1)
- Partner loyalty exchange (complex)

13. Notifications

Frontend pages: admin/notifications **What I'll build (partially):**

- Email notifications (order confirmation, status updates via Spring Mail)
- In-app notification store (DB-backed, list + mark read)
- Push notifications / FCM (complex)
- WhatsApp/SMS notifications (3rd party)

14. 📈 Reports (Basic Only)

Frontend pages: admin/reports **What I'll build (partially):**

- Sales report (total revenue, orders, average order value by date range)
- Export as CSV
- admin/reports/gst, admin/reports/cohort, admin/reports/revenue-analytics (complex BI)
- admin/reports/abandoned-cart, admin/reports/promotion-roi

❑ SKIPPED — Cannot Build in 1 Week

These are either too complex, need 3rd party infrastructure, or are AI/ML features.

Feature	Frontend Page	Why Skipping
Live GPS Delivery Tracking	admin/delivery/live-tracking	Needs mobile app for delivery partners + WebSocket GPS stream
Route Optimization	admin/delivery/route-optimization	Needs Google Maps API + complex algorithm
Fleet Management	admin/delivery/fleet	Needs delivery partner mobile app
Delivery Partner App	admin/delivery/partners	Separate mobile app needed
Vendor Portal	admin/vendors, admin/vendors/settlements	Full separate portal — 2-3 weeks
Vendor Onboarding	admin/vendors/onboarding	Complex KYC + document workflow
AI Recommendations	account/page (recommendations tab)	ML model — months of work
Demand Forecasting	admin/inventory/forecast	ML/Python analytics pipeline
Fraud Detection	admin/customers/fraud-detection	ML risk scoring
Customer Segmentation	admin/customers/segmentation	BI/analytics
Cohort Analysis	admin/reports/cohort	BI/analytics
A/B Testing	admin/promotions/ab-testing	Complex experiment framework
Campaign Builder	admin/promotions/campaign-builder	Drip email/WhatsApp sequences
GST Filing Report	admin/reports/gst	Complex GSTR-1/2/3B format
SEO Management	admin/products/seo	Nice-to-have, not business-critical
WhatsApp OTP/Ordering	auth, checkout	3rd party integration, complex
Social Login	auth	Google/Apple OAuth setup

Feature	Frontend Page	Why Skipping
Wishlist / Gift Cards	account/wishlist, account/gift-cards	Low priority
Community Lists	community-lists-store.ts	Low priority feature
Offline Mode	offline store	PWA complexity

Part 3: 1-Week Delivery Plan

What You Get in 1 Week: 11 Fully Working API Modules

Day 1: Project setup + Database schema + Auth APIs (Login, JWT, OTP)
 Day 2: Product Catalog + Categories APIs
 Day 3: Cart + Promotions/Coupons + Loyalty (basic)
 Day 4: Orders/Checkout + Customer Profile + Addresses
 Day 5: Admin Dashboard + Admin Products (CRUD + Bulk Import)
 Day 6: Admin Orders + Admin Inventory + Admin Customers
 Day 7: Notifications (email) + Reports (basic) + Testing + Swagger docs

Part 4: What Happens After Week 1

These can be built in **Week 2–4** if the client wants:

Feature	Timeline
Payment Gateway (Razorpay)	+2 days
Delivery Partner basic management	+3 days
Vendor Portal (basic)	+1 week
Loyalty points burn at checkout	+1 day
GST Report	+2 days
Social Login (Google)	+2 days
Live Order Tracking (WebSocket)	+3 days
Billing Software Integration	+3 days

Summary for Client

	Spec (Full)	Week 1 Delivery
Architecture	12 Microservices	1 Monolith (scalable)
Databases	5 (PostgreSQL, MongoDB, Redis, Elasticsearch, ClickHouse)	1 (PostgreSQL)

	Spec (Full)	Week 1 Delivery
Infrastructure	Kubernetes + Kafka + AWS EKS	Single server
Auth	OTP + OAuth + WhatsApp	Email OTP + JWT
Products	Full catalog + Elasticsearch	Full catalog + DB search
Cart	Redis-based	DB-based
Orders	Full lifecycle + GPS	Full lifecycle (no live GPS)
Admin Dashboard	Real-time BI	Working dashboard
Delivery	Fleet + GPS + Route AI	Basic partner assignment only
Vendors	Full portal	Skipped
AI/ML	6 ML models	Skipped
Notifications	Push + WhatsApp + SMS	Email only
Total APIs	80+ endpoints	~50 endpoints
Frontend connected?	Yes	Yes — full frontend works

Bottom line: The frontend will be fully functional and usable after Week 1. All 14 frontend pages that matter for the business (product browsing, cart, checkout, orders, admin panel) will have real working backend APIs. Skipped features use existing mock data and can be wired later.